ESCUELA DE INGENIERÍA ELÉCTRICA

PONTIFICIA UNIVERSIDAD CATÓLICA DE VALPARAÍSO

Guía 5

Trabajo con ServoMotor

Curso: EIE390 - Robótica e Inteligencia Artificial
Profesor: Guillermo Cid Ampuero

Objetivos

- Objetivo 1: Comprender los fundamentos de la Modulación por Ancho de Pulso (PWM) y su relación con el control de posición angular en servomotores.
- Objetivo 2: Implementar programación orientada a objetos en MicroPython para la generación y manipulación de señales PWM utilizando la Raspberry Pi Pico.
- Objetivo 3: Desarrollar un sistema de control práctico que permita posicionar físicamente un actuador en ángulos específicos mediante la conversión de variables matemáticas a ciclos de trabajo (duty cycle).

Repositorio del Curso

1 Conceptos Previos: Electrónica Básica para Actuadores

1.1 Conexión de un Servomotor

Un servomotor es un actuador rotativo que permite un control preciso de su posición angular. A diferencia de un motor de corriente continua tradicional que gira indefinidamente, un servo estándar suele moverse dentro de un rango específico (generalmente de 0° a 180°) dependiendo de la señal de Modulación por Ancho de Pulso (PWM) que reciba.

Para funcionar correctamente e interactuar con la Raspberry Pi Pico, el servomotor cuenta con tres cables de conexión, los cuales se identifican universalmente por sus colores:

- **GND (Tierra):** Correspondiente al cable de color marrón (o negro en algunos modelos). Debe conectarse a un pin de tierra (GND) del microcontrolador para cerrar el circuito.
- **VCC (Alimentación):** Correspondiente al cable de color rojo. Provee la energía necesaria para el torque del motor. Se recomienda conectarlo a una fuente de 5V (como el pin VBUS de la placa).
- **SIGNAL (Señal):** Correspondiente al cable de color naranja (o amarillo/blanco). Es el canal por donde el microcontrolador envía los pulsos PWM que dictan el ángulo exacto al que debe posicionarse el motor.

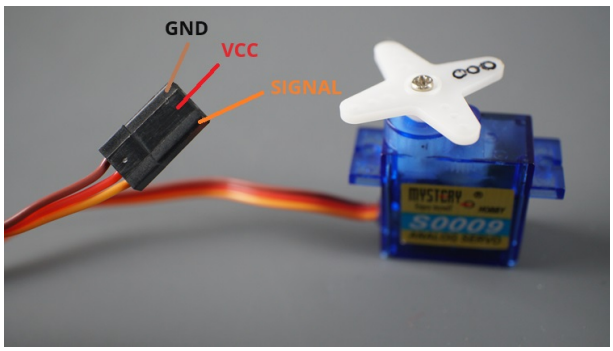

Figura 1: Identificación de los terminales de Tierra (GND), Alimentación (VCC) y Señal en un servomotor estándar.

Para controlar el servomotor, puedes conectar el pin de señal (datos) a cualquier GPIO que pueda generar señales PWM. Todos los pines marcados en verde claro en el diagrama de pines pueden generar señales PWM (que son prácticamente todos los GPIO). Lo conectaremos al GPIO 0, pero puedes usar cualquier otro GPIO siempre que especifiques el pin correcto en el código.

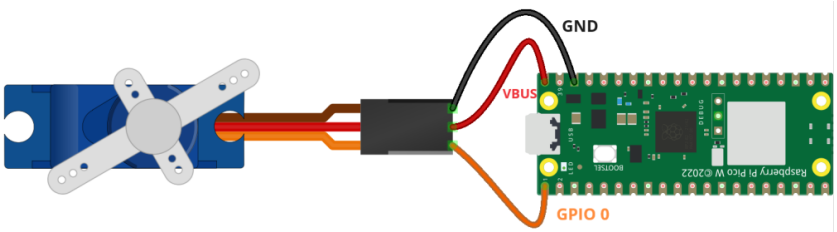

Figura 2: Conexión con Raspberry Pi Pico

Al usar un servo pequeño como el S0009 (o SG90), puede alimentarlo directamente desde el pin de alimentación Pico de la Raspberry Pi. Sin embargo, para otros modelos, es posible que necesite una fuente de alimentación externa (consulte la hoja de datos de su servo).

Cable	**Color**	**Raspberry Pi Pico**
Alimentación/VCC	Rojo	5V (VBUS)
GND	Negro o marrón	GND
Señal	Amarillo, naranja o blanco	GPIO 0 (o cualquier otro pin PWM)

Cuadro 1: Resumen de conexiones de los pines del servomotor.

El eje del servomotor se puede posicionar en distintos ángulos, desde 0° hasta 180° (según el modelo). Los servomotores se controlan mediante una señal de modulación por ancho de pulso (PWM). La señal PWM enviada al motor determina la posición del eje.

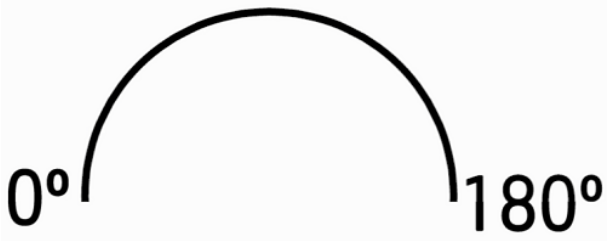

Figura 3: Grados del ServoMotor

Para controlar el motor, basta con utilizar las capacidades PWM de la Raspberry Pi Pico enviando una señal de 50 Hz con el ancho de pulso adecuado. Por lo general, los servomotores S0009 o SG90 (capaces de 180º), el que estamos usando en este tutorial, utilizado en la mayoría de los proyectos de microcontroladores, tienen un rango de ancho de pulso de alrededor de 550 a 2400 microsegundos :

- Ancho mínimo de pulso (0º): alrededor de 550 microsegundos.
- Ancho máximo de pulso (180º): alrededor de 2400 microsegundos

2 Actividad 1: Control de Servomotor con Señales PWM

2.1 Objetivo

Comprender el control de un servomotor mediante la generación de señales PWM (Modulación por Ancho de Pulso) utilizando programación orientada a objetos en MicroPython. El estudiante conectará el actuador e implementará una arquitectura de dos archivos: una librería externa (`servo.py`) que maneja la lógica de hardware, y un script principal que la importa para ejecutar posiciones angulares.

2.2 Materiales Requeridos

- 1x Raspberry Pi Pico
- 1x Protoboard
- 1x Servomotor (ej. SG90)
- Cables Jumper

2.3 Instrucciones

1. **Montaje del circuito:** Conecte los pines del servomotor directamente a la Raspberry Pi Pico siguiendo esta distribución:
   - Cable **Marrón o Negro (GND)**: Conectar a cualquier pin **GND** de la Pico (ej. Pin físico 38).
   - Cable **Rojo (VCC)**: Conectar al pin **VBUS** (Pin físico 40) para asegurar los 5V necesarios para el torque del motor.
   - Cable **Naranja o Amarillo (Señal)**: Conectar al pin **GP0** (Pin físico 1), que será el encargado de enviar la señal PWM.
2. **Arquitectura del código:** Para este ejercicio utilizaremos dos archivos. El primero es `servo.py`, el cual funciona como una librería que abstrae la complejidad matemática de la conversión de ángulos a ciclos de trabajo (duty cycle). El segundo archivo será nuestro programa principal que invocará estas funciones.
3. **Carga de la librería:** Copie el código de la librería proporcionada. En Thonny, guarde este archivo **directamente en la memoria de la Raspberry Pi Pico** con el nombre exacto de `servo.py`.
4. **Carga del programa principal:** Abra una nueva pestaña en Thonny, copie el código del script principal (que importa la clase `Servo`) y guárdelo en la Pico o en su computadora (ej. `main.py`).
5. **Prueba de funcionamiento:** Ejecute el script principal. Observe cómo el motor realiza una secuencia de movimientos posicionándose en 0°, 90° y 180°. Para finalizar, presione `Ctrl+C` en la consola; esto asegura que la señal PWM se detenga correctamente, protegiendo el motor.

- **Codigo 1: ejemplo_1.py**

3 Actividad 2: Automatización de Movimientos mediante Ciclos For

3.1 Objetivos

El propósito de esta actividad es que el estudiante logre integrar estructuras de control iterativas con el manejo de hardware físico. Se busca que comprendan los fundamentos de la Modulación por Ancho de Pulso (PWM) aplicados a un barrido automático, utilizando la función `range()` para definir trayectorias angulares y automatizar el posicionamiento del servomotor sin intervención manual constante.

3.2 Fundamento Teórico: El ciclo `for` y la función `range`

Para lograr que el servomotor recorra todas las posiciones de forma gradual, utilizaremos un bucle iterativo. En Python, la función `range(inicio, fin, paso)` es la herramienta estándar para generar secuencias numéricas. Recuerde que el parámetro `fin` es exclusivo, por lo que debe ajustar el límite superior para alcanzar el ángulo deseado. Además, el `paso` determinará la resolución del movimiento del motor (qué tan "saltado.º fluido se verá).

3.3 Instrucciones y Desafío de Programación

1. **Preparación:** Utilice como base el archivo realizado en la Actividad 1. Asegúrese de que la librería `servo.py` permanezca en la memoria de la Raspberry Pi Pico.
2. **Reto de Código:** Deberá crear un nuevo script que realice un barrido automático (sweep) desde **0° hasta 180°**, incrementando la posición de **10 en 10 grados** en cada paso. Entre cada movimiento, el programa debe esperar **1 segundo** y mostrar en la consola el ángulo actual mediante el método `get_current_angle()`.
3. **Estructura sugerida:** A continuación se presenta la estructura base. Usted debe completar la lógica del ciclo dentro del bloque `try`:

```
from servo import Servo
from time import sleep

# Instanciar el servo en el pin correspondiente
servo = Servo(pin=0)

try:
    print("Iniciando barrido automático...")
    while True:
        # --- ESCRIBA SU CÓDIGO AQUÍ ---
        # Implemente un ciclo for que recorra de 0 a 180 grados
        # Use servo.move() y sleep()
        pass
        # ------------------------------

except KeyboardInterrupt:
    print("\nPrograma detenido por el usuario")
    servo.stop()
```

- **Codigo 2: ejemplo_2.py**

4 Actividad 3: Control Interactivo por Comunicación Serial

4.1 Objetivos

El objetivo de esta actividad es establecer una comunicación bidireccional en tiempo real entre el computador y la Raspberry Pi Pico. El estudiante deberá capturar comandos de texto ingresados por el usuario a través del monitor serial, procesar estas cadenas de caracteres (strings), convertirlas a valores numéricos (integers) y utilizarlos para controlar la posición exacta del servomotor, implementando además rutinas de manejo de errores para evitar fallos en el programa.

4.2 Fundamento Teórico: Lectura de Datos y Manejo de Excepciones

Hasta ahora, hemos preprogramado los movimientos del motor. Para lograr que el sistema responda a comandos ingresados por el usuario en tiempo real (como escribir '90' en la consola de Thonny), necesitamos usar la librería estándar `sys`.

A continuación, se analizan las tres herramientas de programación clave que deberá incorporar en su código:

1. Captura y limpieza de datos seriales:

```
import sys
comando = sys.stdin.readline().strip()
```

La función `sys.stdin.readline()` lee todo lo que el usuario escribe en la consola hasta que presiona la tecla *Enter*. Sin embargo, esta lectura incluye caracteres invisibles de salto de línea (como `\n`). El método `.strip()` se encarga de limpiar estos espacios y saltos de línea al inicio y al final del texto, entregando una cadena de texto (string) limpia.

2. Conversión de Tipos de Datos (Casting):

```
angulo = int(comando)
```

Todo lo que ingresa por la consola serial es interpretado por Python como texto (String). El hardware del servo no entiende de letras, requiere números. La función `int()` transforma ese texto numérico en un número entero válido para la función de movimiento.

3. Manejo de Errores (`try...except`): ¿Qué sucede si el usuario, por error, escribe la palabra 'hola' e intentamos convertirla a un número con `int()`? El programa colapsará inmediatamente mostrando un error de tipo `ValueError`. Para prevenir esto, usamos un bloque de contención:

```
try:
    angulo = int(comando)
    # Aquí iría el código para mover el motor
except ValueError:
    print("Error: Por favor, ingrese solo números.")
```

Si la conversión falla, el programa 'atrapa' el error en el bloque `except`, imprime el mensaje de advertencia y continúa funcionando sin detener la ejecución de la placa.

4.3 Instrucciones

1. Utilice el código a continuación , para poder controlar el servomotor desde la terminal.
2. Prueba el código con comandos erróneos, para que observe como responden las excepciones al momento de generarse un error.

- **Codigo 3: ejemplo_3.py**

5 Actividad 4: Control Multieje con Múltiples Servomotores

5.1 Objetivos

El objetivo de esta actividad es escalar el sistema de control a múltiples actuadores, sentando las bases para proyectos de robótica más complejos (como brazos robóticos o sistemas pan-tilt). El estudiante aprenderá a instanciar múltiples objetos a partir de una misma clase y a diseñar un protocolo de comunicación serial básico, procesando cadenas de texto complejas para enrutar comandos específicos a distintos motores de forma independiente y concurrente.

5.2 Materiales Requeridos

- 1x Raspberry Pi Pico
- 1x Protoboard
- **2x** Servomotores
- Cables Jumper

5.3 Montaje del Circuito

1. Mantenga el **Servo 1** conectado exactamente como en las actividades anteriores (Señal en **GP0**).
2. Conecte el **Servo 2** de la siguiente manera:
   - **GND:** Al mismo bus de tierra de la protoboard.
   - **VCC:** Al bus de 5V (conectado a VBUS). *Precaución: Al usar dos motores, es estrictamente necesario que ambos estén alimentados desde los 5V para evitar reinicios por falta de corriente en la placa.*
   - **Señal:** Conecte el cable de señal (naranja/amarillo) al pin **GP1** (Pin físico 2).

5.4 Fundamento Teórico: Instancias Múltiples y Procesamiento de Texto

Para controlar dos motores de forma independiente mediante la consola, ya no basta con enviar un solo número. Necesitamos establecer un formato o protocolo, por ejemplo: `motor,angulo` (escribir `"1,90"` para mover el motor 1 a 90°, o `"2,45"` para el motor 2).

Para lograr esto en Python, introduciremos dos conceptos clave:

1. Múltiples Objetos (Instancias): Gracias a que utilizamos la clase `Servo` en nuestra librería, no necesitamos reescribir la matemática para el segundo motor. Simplemente creamos un segundo .ºbjeto.ªsignándole un pin distinto:

```
servo1 = Servo(pin=0)
servo2 = Servo(pin=1)
```

2. Separación de Cadenas de Texto (`split`): Cuando el usuario escriba `"1,90"`, recibiremos un solo texto. Para separar el identificador del motor del ángulo deseado, usamos el método `.split(',')`, el cual divide el texto original usando la coma como separador y nos entrega una lista:

```
comando = "1,90"
partes = comando.split(",") # Genera la lista: ["1", "90"]
motor_id = int(partes[0])   # Extrae el 1
angulo = int(partes[1])     # Extrae el 90
```

5.5 Instrucciones y Desafío de Programación

1. **El Reto:** Modifique el programa interactivo de la Actividad 5 para que soporte ambos motores. El programa debe advertir al usuario si el formato ingresado es incorrecto, si el número de motor no existe (debe ser 1 o 2), o si el ángulo está fuera de los límites (0° a 180°).

2. **Estructura sugerida:** A continuación se presenta la base del sistema.

```
from servo import Servo
import sys

# Instanciar ambos motores
servo1 = Servo(pin=0)
servo2 = Servo(pin=1)

print("Formato requerido: motor,angulo (Ejemplo: 1,90 o 2,180)")

try:
    while True:
        cmd = sys.stdin.readline().strip().lower()
        if cmd == 'salir':
            break

        try:
            # Separar el comando
            partes = cmd.split(',')

            # --- ESCRIBA SU CÓDIGO AQUÍ ---
            # 1. Valide que 'partes' tenga exactamente 2 elementos (len)
            # 2. Convierta las partes a enteros (motor_id y angulo)
            # 3. Use condicionales (if/elif) para mover servo1 o servo2
            pass
            # ------------------------------

        except ValueError:
            print("Error: Ingrese solo números separados por coma.")

except KeyboardInterrupt:
    print("\nPrograma detenido.")

finally:
    # Asegurar que ambos motores se detengan
    servo1.stop()
    servo2.stop()
```

3. **Resultado:** Su código debería ingresa como ejemplo `1,0`, luego `2,180`, y finalmente intente ingresar un formato erróneo como `3,90` o `hola` para demostrar que su sistema no colapsa.

- **Codigo 4: ejemplo_4.py**

6 Referencias

1 Link 1

2 Link 2

3 Link 3

4 Link 4